Hito 2 — Kernels CUDA: reducción (suma) en GPU

Programación Concurrente · Laura Elizabeth Giménez y Oscar Gabriel Sosa · 21/10/2026

1. Problema y versiones

Se resuelve la misma tarea del Hito 1: sumar un vector de N float32 con todos los valores en 1.0 (suma esperada = N), ahora en GPU con Numba CUDA. Se comparan la CPU secuencial del Hito 1, un kernel ingenuo y un kernel optimizado.

- **Ingenuo:** cada hilo hace cuda.atomic.add sobre un único acumulador en memoria global.
- **Optimizado:** carga coalescida a memoria compartida y reducción en árbol (8 pasos con 256 hilos, con syncthreads); cada bloque deja un parcial y el host los suma en float64.

Medición: tiempo de kernel con cuda.event, calentamiento (compilación JIT) descartado y mediana de 9 repeticiones, con 256 hilos por bloque. Entorno: Google Colab, Tesla T4 (cc 7.5), CUDA 13.0, Numba 0.61.2.

2. Resultados

N	CPU (ms)	Ingenua (ms)	Optimizada (ms)	opt/ing	opt/CPU	GB/s ing	GB/s opt	H2D (ms)
2^20	0.640	3.608	0.111	32.6×	5.78×	1.2	37.8	1.315
2^22	2.680	14.274	0.287	49.8×	9.35×	1.2	58.6	3.741
2^24	12.056	33.282	0.624	53.4×	19.34×	2.0	107.6	15.320

Todos los resultados validaron (suma = N). GB/s = 4·N bytes / tiempo. opt/CPU compara solo el kernel, sin la copia H2D.

Hilos por bloque (optimizado, N = 2^24): 128 → 0.598 ms (112.2 GB/s); 256 → 0.639 ms (105.0 GB/s); 512 → 0.703 ms (95.5 GB/s).

Nsight Compute (N = 2^24; el ingenuo se perfiló con el conjunto basic porque con full la corrida falló):

Métrica	Ingenua	Optimizada
Memory Throughput	2.06 %	83.51 %
DRAM Throughput	0.51 %	19.32 % (61.76 GB/s)
Compute (SM) Throughput	1.22 %	83.51 %
Ocupación teórica / lograda	100 % / 87.63 %	100 % / 88.72 %
Duración bajo ncu	58.87 ms	1.52 ms

ncu fija los relojes de la GPU, por eso sus duraciones no son comparables con las de cuda.event; se usan para ver el cuello de botella.

3. Análisis

- **El atomicAdd sobre una sola dirección serializa.** El ingenuo es de 32× a 53× más lento que el optimizado y la brecha crece con N. Usa solo 2 % de la memoria y 1.2 % del cómputo aunque su ocupación es alta (87.6 %): hay muchos warps, pero esperando su turno sobre la misma dirección. El problema es la contención, no la falta de hilos.
- **El optimizado usa mejor la GPU.** Llega a 107.6 GB/s (cerca de un tercio de los ~320 GB/s del pico de la T4) y mejora con N porque los costos fijos de lanzamiento pesan menos. Con ncu no está limitado por DRAM (19.3 %) sino que la memoria on-chip y el cómputo están al 83.5 %. Hipótesis, no verificada: el cuello de botella es el trabajo dentro del bloque (memoria compartida y syncthreads). Una mejora posible, no probada: que cada hilo sume varios elementos en un registro antes del árbol.
- **CPU vs. GPU.** Solo con el kernel, la GPU gana 19.3× (0.624 vs. 12.056 ms). Pero copiar el vector tarda 15.320 ms, más que sumarlo en la CPU: el total del optimizado (kernel + parciales + suma final, 0.819 ms) más la copia da ≈ 16.1 ms contra 12.1 ms. Para una suma aislada no conviene mover los datos; la GPU se justifica si los datos ya están en ella o hay mucho más cómputo por dato.
- **Tamaño de bloque.** 128 hilos fue el más rápido y 512 el más lento (~18 % más). Entre 128 y 256 la diferencia (6.8 %) es comparable al ruido entre corridas (~2.5 %), así que no es concluyente.
- **Precisión float32.** Con N = 2^25 el ingenuo devolvió 16 777 216 en vez de 33 554 432 (checksum inválido): los enteros en float32 son exactos solo hasta 2^24 y sumar 1.0 a ese valor no cambia el acumulador. El optimizado dio el resultado exacto porque cada bloque suma 256 elementos y el total se junta en float64.

4. Conexión con la teoría y limitaciones

El modelo SIMT y los warps (Clase 5) explican por qué muchos hilos sobre la misma dirección no suman rendimiento; la jerarquía de memoria, la memoria compartida y la coalescencia (Clase 6) explican la mejora del optimizado; la reducción en árbol y el perfilado (Clase 7) permiten ubicar el cuello de botella. Limitaciones: una sola GPU (T4) y una sola sesión de Colab; la variación entre corridas es de unos pocos por ciento (un script suelto dio 1.342 ms para el optimizado, contra 0.624 ms en el benchmark con calentamiento); las hipótesis marcadas como tales no se verificaron con más métricas.